TP – Services Avancés

PLAN DU COURS

- Séance 8 : **TP8** : noté en classes : Exposition des **CRUD** simples liés au diagramme de classe Projet-ProjetDetail-Equipe (Développement + Tests)

- Séance 8 + 9 : **TP9** : **Affectations et Desaffectations** liés au classe Projet-ProjetDetail-Equipe (Développement + Tests)

- Séance 12 : **TP12** : **Affectations et Desaffectations** liés au projet Foyer (Développement + Tests) + **Kewords** + **JPQL**

TP8 – En Classe (Séance 8)

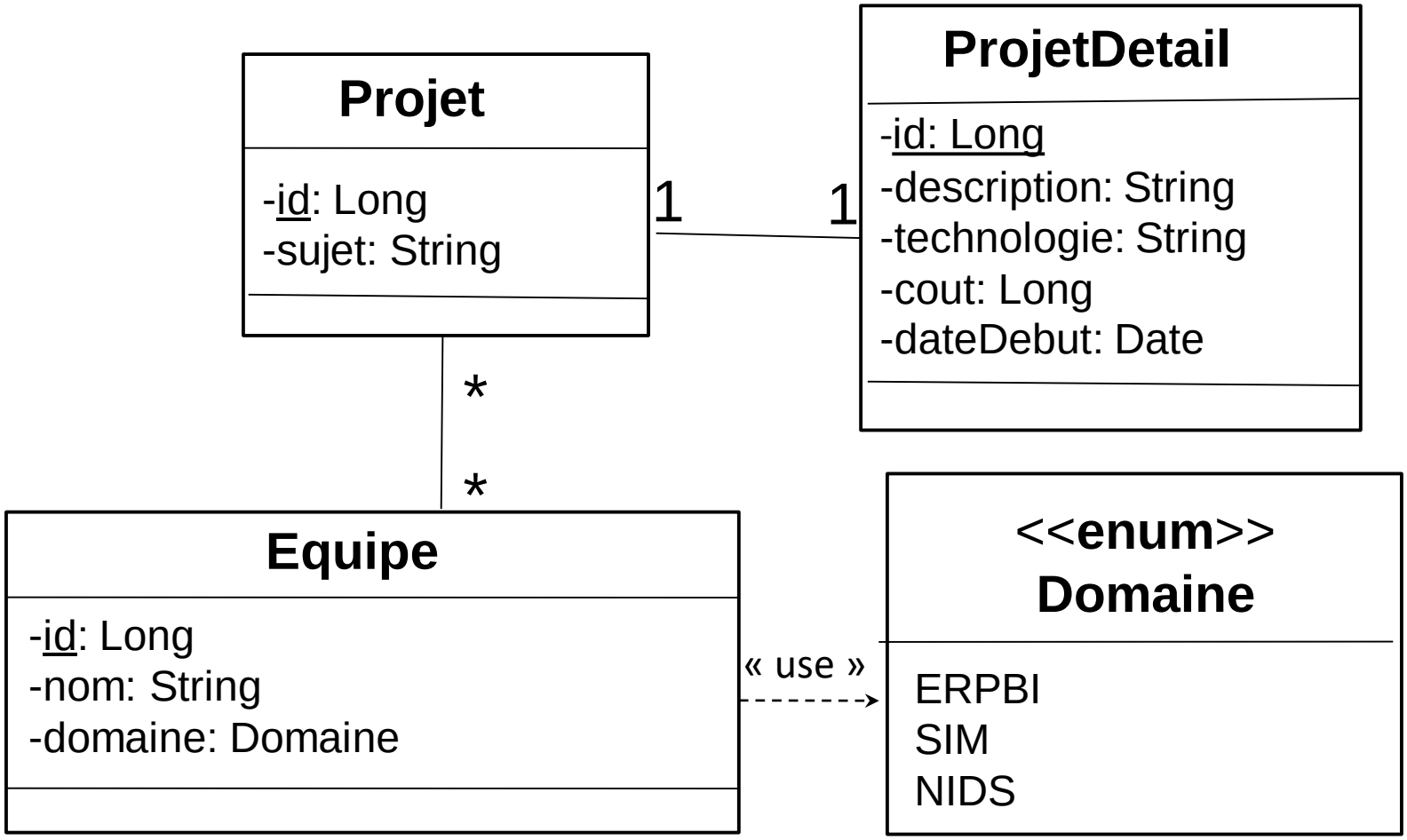
Nous allons développer une nouvelle application "**tp8-prenom-nom-classe**". C'est une application simplifiée de Gestion de projet.
- Dans le support de cours "8- Services Avancés (Affectations + Désaffectations).pdf", page suivante, il y a le diagramme de classe de cette application Projet / ProjetDetail / Equipe

- Créer un projet **Spring Boot " tp8-prenom-nom-classe"** (suivre les étapes décrites dans le cours Spring Boot)
- Créer les **Entités** Projet / ProjetDetail / Equipe. Créer l'**Enum** Domaine
- Créer les **Associations** nécessaires
- Créer les **Repository**
- Créer les **Services et les Interfaces**
- Créer les **Controllers**

- Important: Vu l'importance de ce homework et la complexité du scénario de test des méthodes avancées (affectations+ désaffectation), ne mettez pas de captures de ce homework sur le classroom. La validation sera faite en classe, présence obligatoire donc svp, sinon pas de note pour ce homework, c'est considéré comme un test. Très bon courage

Affectation

Soit le diagramme de classes suivant:
Ce diagramme sera un exemple pour comprendre le mécanisme d'Affectations.
Dans la relation **bidirectionnelle** Projet-ProjetDetail, **ProjetDetail est le fils**.
Dans la relation **bidirectionnelle** Equipe-Projet, **Projet est le fils.**



Cas 1 : Affectation

On souhaite créer le service qui permet d'Ajouter un Projet et un Projet Detail et d'affecter le ProjetDetail au Projet

Solution :
- Dans l'entité Projet (Parent) ont doit cascader les actions du Projet vers le projetDetail
- Dans la méthode, on doit faire un ajout simple :

Cas 1 : Affectation

Service ProjetServiceImpl :

```
public Projet addProjetAndProjetDetailAndAssign(Projet projet) {
    return projetRepository.save(projet);
}
```

Controller ProjetRestController :

http://localhost:8089/tp8/projet/ajouter-projet-et-projet-detail

```
@PostMapping("/ajouter-projet-et-projet-detail")
public Projet addProjetAndProjetDetail(@RequestBody Projet p) {
    Projet projet = projetService.addProjetAndProjetDetailAndAssign(p);
    return projet;
}
```

Cas 1 : Affectation

Entité Projet (**Parent**) : (l'ajout d'un projet provoque l'ajout d'un projet détail grâce à cascade)

```
@OneToOne(cascade = CascadeType.ALL)
  private ProjetDetail projetDetail;
```

Entité ProjetDetail (**fils**) :

```
@OneToOne(mappedBy = "projetDetail")
  @ToString.Exclude
  @JsonIgnore
  private Projet projet;
```

Cas 1 : Affectation

Objet JSON à utiliser : (à l'intérieur de l'objet projet, on met l'objet projet détail) :

```
{
  "sujet": "projet numéro 1",
  "projetDetail": {
    "description": "détail du projet",
    "technologie": "Java",
    "cout_provisoire": 1000,
    "dateDebut": "2022-11-14T23:52:48.343Z"
  }
}
```

Attention si Relation OneToMany ou ManyTomany :

```
{
" attribut1" : …,
" attribut2 " : [
{…} ,
{….}
]
}
```

Cas 1 : Affectation

Résultat dans la base de données :

Table t_projet_detail :

pd_id	pd_cout_provisoire	date_debut	pd_description	pd_technologie
16	0	2022-11-15	string	string

Table t_projet :

projet_id	projet_sujet	projet_detail_pd_id
2	string	16

Cas 2 : Affectation

On souhaite créer le service qui permet d'Affecter un ProjetDetail à un Projet (le ProjetDetail et le Projet sont déjà créés dans la base de données :

Solution :
Dans la méthode, on doit récupérer de la base de données le Projet et le ProjetDétail, puis affecter le ProjetDetail (fils) au Projet (parent), puis faire une mise à jour :

Cas 2 : Affectation

Service :

```
public void assignProjetDetailToProjet(Long projetId, Long projetDetailId) {
        Projet projet = projetRepository.findById(projetId).get();
        ProjetDetail projetDetail = projetDetailRepository.findById(projetDetailId).get();
        // on set le fils dans le parent :
        projet.setProjetDetail(projetDetail);
        projetRepository.save(projet);
    }
```

Controller :

#http://localhost:8089/tp8/projet/affecter-projet-a-projet-details/**1/1**

```
    @PutMapping("/affecter-projet-a-projet-details/{projet-id}/{projet-details-id}")
    public void affecgterProjetAProjetDetail(@PathVariable("projet-id") Long proejtId,
    @PathVariable("projet-details-id") Long proejtDetailsId) {
        projetService.assignProjetDetailToProjet(proejtId, proejtDetailsId);
    }
```

Cas 3 : Affectation

On souhaite créer le service qui permet d'Affecter un Projet à une Equipe (le Projet et l'Equipe sont déjà créés dan la base de données) :

Solution :
Dans la méthode, on doit récupérer de la base de données l'Equipe et le Projet, puis affecter le Projet (fils) à l'Equipe (parent), puis faire une mise à jour :

Cas 3 : Affectation

Service :

```
public void assignProjetToEquipe(Long projetId, Long equipeId) {
        Projet projet = projetRepository.findById(projetId).get();
        Equipe equipe = equipeRepository.findById(equipeId).get();
        // on set le fils dans le parent  :
        equipe.getProjets().add(projet);
        equipeRepository.save(equipe);
    }
```

Attention : Initialiser la liste des Projets dans Equipe.java sinon erreur :

```
@ManyToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    private Set<Projet> projets = new HashSet<Projet>();
```

Or

```
@ManyToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    private List<Projet> projets = new ArrayList<Projet>();
```

Controller (presque le même code que le Controller de l'Affectation 2) :
http://localhost:8089/tp8/projet/affecter-projet-equipe/**1/1**

Cas 4 : Affectation

On souhaite créer le service qui permet d'Ajouter un Projet d'Affecter un Projet Detail à ce projet (Projet Détail est déjà dans la base de donnes) :

Solution :
Dans la méthode, on doit récupérer de la base de données le Projet Detail, puis le setter dans la le Projet (parent), puis faire une mise à jour :

Cas 4 : Affectation

Service :

```
public Projet addProjetAndAssignProjetToProjetDetail(Projet projet, Long projetDetailId) {
      ProjetDetail projetDetail = projetDetailRepository.findById(projetDetailId).get();
      // on set le fils dans le parent :
      projet.setProjetDetail(projetDetail);
      return projetRepository.save(projet);
  }
```

Controller à finir pour avoir l'URL :
http://localhost:8089/tp8/projet/creer-projet-et-affecter-projet-detail-a-projet/1
(le projet sera envoyé dans le body, l'id du projet detail dans le path).

Cas 5 : DésAffectation

On souhaite créer le service qui permet de désaffecter un ProjetDetail d'un Projet (Le Projet et le ProjetDetail sont déjà créés dans la base de données) :

Solution :
Récupérer le Projet de la base de données. Puis, il suffit de setter l'attribut projetDetail à **null** dans Projet, puis faire une mise à jour :

Cas 5 : DésAffectation

Service :

```
public Projet DesaffecterProjetDetailFromProjet(Long projetId) {
    Projet projet = projetRepository.findById(projetId).get();
    projet.setProjetDetail(null);
    return projetRepository.save(projet);
}
```

Controller à finir pour avoir l'URL :
http://localhost:8089/tp8/projet/desaffecter-projet-detail/1

Cas 5 : DésAffectation

Table t_projet_detail :

pd_id	pd_cout_provisoire	date_debut	pd_description	pd_technologie
16	0	2022-11-15	string	string

Table t_projet :

projet_id	projet_sujet	projet_detail_pd_id
2	string	

Cas 6 : DésAffectation

On souhaite créer le service qui permet de désaffecter un Projet d'une Equipe (L'Equipe et le Projet sont déjà créés dans la base de donnes) :

Solution :
Récupérer le Projet et l'Equipe de la base de données. Puis, il suffit d'enlever le projet à l'équipe, puis faire une mise à jour :

Cas 6 : DésAffectation

Service :

```
public void desaffecterProjetFromEquipe(Long projetId, Long equipeId) {
        Projet projet = projetRepository.findById(projetId).get();
        Equipe equipe = equipeRepository.findById(equipeId).get();
        // on enlève le fils du parent :
        equipe.getProjets().remove(projet);
        equipeRepository.save(equipe);
    }
```

Attention : Initialiser la liste des Projets dans Equipe.java sinon erreur :

```
@ManyToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    private Set<Projet> projets = new HashSet<Projet>();
```

Or

```
@ManyToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    private List<Projet> projets = new ArrayList<Projet>();
```

Controller à finir pour avoir l'URL :

http://localhost:8089/tp8/projet/desaffecter-projet-de-equipe/1/1

Important

Attention : Pour pouvoir faire les affectations et désaffectations ci-dessus, on doit injecter les repository nécessaires dans le service ProjetServiceImpl :

```
@Service
@AllArgsConstructor
  public class ProjetServiceImpl implements IProjetService {

    ProjetRepository projetRepository;

    ProjetDetailRepository projetDetailRepository;

    EquipeRepository equipeRepository;
```

Vous devez faire de même pour votre TP (**voir page suivante**), en injectant repository nécessaires (PisteRepository, AbonnementRepository) dans votre Service SkieurRepository :

TP9 – Homework (Séance 9)

Utiliser le projet **tp8-prenom-nom-classe** développé en classe, pour développer et tester les **6 services affectations et désaffectations** vu dans le cours (voir slides ci-dessus).

KEYWORDS

- Spring Data JPA va créer le code pour les méthodes que tu souhaites utiliser. Il suffit de lui indiquer les méthodes à utiliser :

Keyword	Sample	Equivalent to
GreaterThan	findByanneeUniversitaireGreaterThan(Date dateAU);	Select p from Reservation r where r.anneeUniversitaire > :dateAU
LessThan	findByanneeUniversitaireLessThan(Date dateAU);	TODO in class
Between	findByanneeUniversitaireBetween(Date dFrom, Date dTo);	TODO in class
IsNotNull, NotNull	findByNumeroChambreNotNull();	TODO in class
IsNull, Null	findByDescriptionNull();	TODO in class
Like	findByAdresseLike(String add);	TODO in class
Contains	findByPrenomEtudiantAndNomEtudiat(String p, String n);	TODO in class
(No keyword)	Long count();	TODO in class

KEYWORDS

- Afficher **la liste** des chambres qui ont un type donné (double par exemple) .

```
@Repository
public interface ChambreRepository extends JpaRepository<Chambre, Long>
{
    List<Chambre> findAllByTypeChambre(TypeChambre tc);
}
```

KEYWORDS

- Afficher **la** chambre qui a un numéro donné.

```
@Repository
public interface ChambreRepository extends JpaRepository<Chambre, Long>
{
    Chambre findChambreByNumeroChambre(long numC);
}
```

JPQL

- **JPQL** = Java Persistence Query Language
- JPQL peut être considéré comme une version orientée objet de SQL.
- En JPQL, on sélectionne des objets d'une entité (une classe annotée par @Entity) en utilisant les noms des attributs et le nom de la classe **de l'entité** en question et non plus ceux de la table de base de données et ses colonnes.

JPQL : SELECT

- Ces méthodes permettent de récupérer les entreprises avec une adresse donnée :
- **JPQL :**

```
@Query("SELECT e FROM Entreprise e WHERE e.adresse =:adresse")
List<Entreprise> retrieveEntreprisesByAdresse(@Param("adresse") String adresse);
```

C'est équivalent à :

```
@Query("SELECT e FROM Entreprise e WHERE e.adresse = ?1")
List<Entreprise> retrieveEntreprisesByAdresse(String adresse);
```

Supposons que nous avons mapper l'entité Entreprise avec **la table associé T_Entreprise**

- **Native Query (SQL et non JPQL) :**

```
@Query(value = "SELECT * FROM T_Entreprise e WHERE e.entreprise_adresse = :adresse",
nativeQuery = true)
List<Entreprise> retrieveEntreprisesByAdresse(@Param("adresse") String adresse);
```

C'est équivalent à :

```
@Query(value = "SELECT * FROM T_Entreprise e WHERE e.entreprise_adresse = ?1" ,
nativeQuery = true)
List<Entreprise> retrieveEntreprisesByAdresse( String adresse);
```

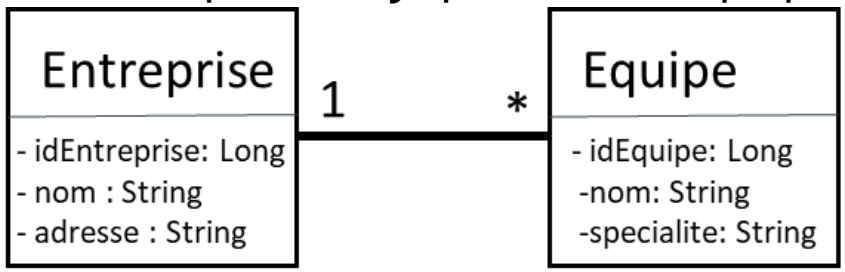
JPQL : SELECT

- Ces méthodes permettent de récupérer les entreprises qui ont une équipe avec une spécialité donnée :
- **JPQL :**

```
@Query("SELECT entreprise FROM Entreprise entreprise , Equipe equipe where
entreprise.id = equipe.entreprise.id and equipe.specialite =:specialite")

List<Entreprise> retrieveEntreprisesBySpecialiteEquipe(@Param("specialite")
String specialite);
```

- **Native Query (SQL et non JPQL) :**

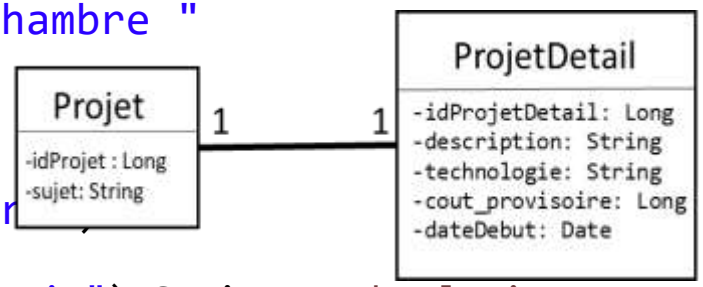
```
@Query(value = "SELECT * FROM T_ENTREPRISE entreprise INNER JOIN T_EQUIPE equipe
ON entreprise.ENTREPRISE_ID = equipe.ENTREPRISE_ENTREPRISE_ID where
equipe.EQUIPE_SPECIALITE =:specialite", nativeQuery = true)

List<Entreprise> retrieveEntreprisesBySpecialiteEquipe(@Param("specialite")
String specialite);
```



JPQL : SELECT

- Cette méthode permet d'afficher les projets qui ont un coût supérieur à coût donné dt et une technologie donnée.
- **JPQL :**

```
@Query("SELECT projet FROM Projet projet, Chambre detail where "
            + "detail.idChambre = projet.projetDetail.idChambre "
            + "and detail.technologie =:technologie "
            + "and detail.cout_provisoire >:cout_provisoir
```




```
List<Projet> retrieveProjetsByCoutAndTechnologie(@Param("technologie") String technologie,
            @Param("cout_provisoire") Long cout_provisoire);
```

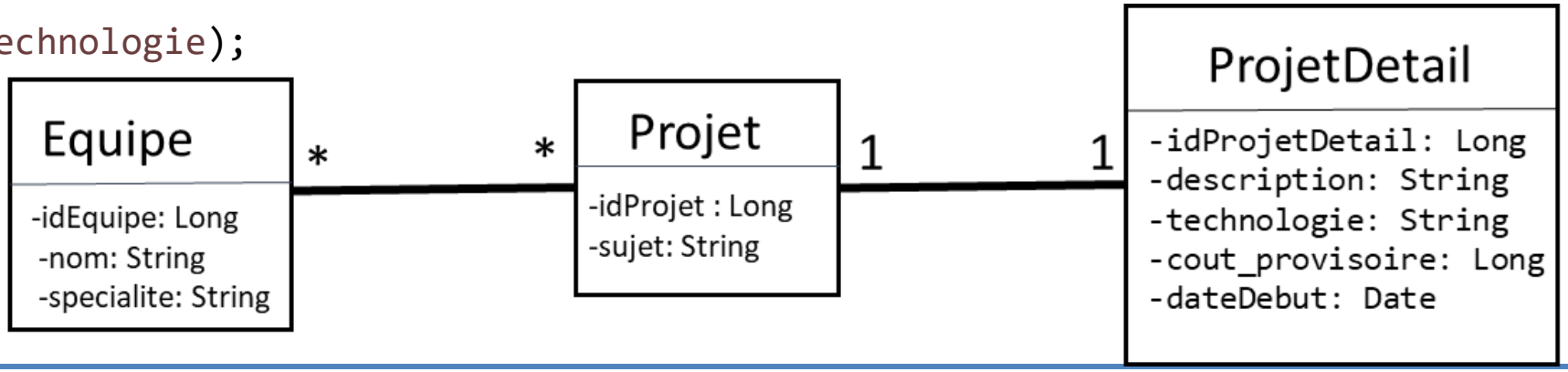
- **Native Query (SQL et non JPQL) :**

```
@Query(value = "SELECT * FROM T_PROJET projet INNER JOIN T_PROJET_DETAIL detail ON
detail.PD_ID =   projet.PROJET_DETAIL_PD_ID WHERE detail.PD_TECHNOLOGIE = ?1 and
detail.PD_COUT_PROVISOIRE > ?2", nativeQuery = true)
    List<Projet> retrieveProjetsByCoutAndTechnologie(@Param("technologie") String technologie,
            @Param("cout_provisoire") Long cout_provisoire);
```

JPQL : SELECT

- Cette méthode permet d'afficher les équipes qui travaillent sur une technologie donnée dont le projet n'a pas encore commencé.
- **JPQL :**

```
@Query("SELECT equipe FROM Equipe equipe"
        + " INNER JOIN equipe.projets projet"
        + " INNER JOIN Chambre detail"
        + " ON detail.idChambre = projet.projetDetail.idChambre"
        + " where detail.dateDebut > current_date"
        + " and detail.technologie =:technologie")
List<Equipe> retrieveEquipesByProjetTechnologie(@Param("technologie")
String technologie);
```



JPQL : UPDATE

- Si nous souhaitons faire un **UPDATE, DELETE et INSERT**, nous devons ajouter l'annotaion **@Modifying** pour activer la modification de la base de données.

- Cette méthode permet de mettre à jour l'adresse de l'entreprise.

- **JPQL :**

```
@Modifying

@Query("update Entreprise e set e.adresse = :adresse where e.idEntreprise =
:idEntreprise")

int updateEntrepriseByAdresse(@Param("adresse") String adresse,
@Param("idEntreprise")

Long idEntreprise);
```

JPQL : DELETE

- Cette méthode permet de supprimer les entreprises qui ont une adresse donnée :

- **JPQL :**

```
@Modifying
@Query("DELETE FROM Entreprise e WHERE e.adresse= :adresse")
int deleteEntreprisebyadresse(@Param("adresse") String adresse);
```

C'est équivalent à :

```
@Modifying
@Query("DELETE FROM Entreprise e WHERE e.adresse= ?1")
int deleteFournisseurByCategorieFournisseur(String adresse);
```

JPQL : INSERT

- Cette méthode permet d'insérer des projets dans la table T_Projet:

- **JPQL :** Nous utilisons Spring Data JPA. Or INSERT ne fait pas partie des spécifications JPA. Donc, nous sommes obligés d'utiliser les Natives Query pour le INSERT.
- **Pas de JPQL pour les requêtes INSERT.**

- **Native Query (SQL et non JPQL) :**

```
@Modifying
@Query(value = "INSERT INTO T_Projet(projet_sujet) VALUES (:projetsujet)",
   nativeQuery = true)
void insertProjet(@Param("projetsujet") String projetsujet);
```

TP12 – Homework (Séance 12)

Utiliser le projet **tp-foyer** développé en classe, pour développer et tester les **services affectations et désaffectations** suivants :

- Créer un Bloc et son Foyer en même temps.
- Affecter un Bloc à un Foyer (Les deux sont déjà dans la base de données).
- Désaffecter un Bloc de son Foyer

- **KeyWords** : Créer un Service planifié qui tourne toutes les 2 minutes, qui affiche la liste des blocs non affectés à aucun foyer.

- **JPQL** : Créer un Service qui permet de trouver la chambre d'un étudiant donné (selon le CIN de l'étudiant)

TP12 – Keywords

Indication pour la méthode avec Keywords :

Dans le Repo : List<Bloc> findAllByFoyerIsNull();

Ajouter 2 blocs, un avec Foyer et un sans Foyer (n'oubliez pas le **cascade** dans l'entité père vers le fils)

@ManyToOne(**cascade = CascadeType.*ALL***)

Foyer foyer;

```
{
  "nomBloc": "bloc A",
  "capaciteBloc": 101,
  "foyer": {
    "nomFoyer": "F1",
    "capaciteFoyer": 1001
  }
}
```

Puis :

```
{
  "nomBloc": "bloc B",
  "capaciteBloc": 105
}
```

TP12 – Keywords

GET /bloc/trouver-blocs-sans-foyer

récupérer toutes les blocs sans foyer

Parameters

No parameters

Execute

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8089/tpfoyer/bloc/trouver-blocs-sans-foyer' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8089/tpfoyer/bloc/trouver-blocs-sans-foyer
```

Server response

Code	Details
200	Response body


```
[
  {
    "idBloc": 3,
    "nomBloc": "bloc B",
    "capaciteBloc": 105,
    "foyer": null,
    "chambres": []
  }
]
```

TP12 – JPQL

Indication pour la méthode JPQL :
Pour préparer le test ajouter **une chambre avec une réservation et avec un étudiant en même temps** (n'oubliez pas l'annotation **cascade** aux bon endroits) :

```
@OneToMany(cascade = CascadeType.ALL)
@ToString.Exclude
@JsonIgnore
Set<Reservation> reservations;

@ToString.Exclude
@JsonIgnore
@ManyToMany(cascade = CascadeType.ALL)
Set<Etudiant> etudiants;
```

TP12 – JPQL

Indication pour la méthode JPQL : Pour préparer le test ajouter **une chambre avec une réservation et avec un étudiant en même temps** :


```
{
  "numeroChambre": 105,
  "typeChambre": "SIMPLE",
  "reservations": [
    {
      "idReservation": "Res-123",
      "anneeUniversitaire": "2023-11-26T14:05:14.824Z",
      "estValide": true,
      "etudiants": [
        {
          "nomEtudiant": "ASKRI",
          "prenomEtudiant": "Mohamed",
          "cinEtudiant": 123321,
          "dateNaissance": "2023-11-26T14:05:14.824Z"
        }
      ]
    }
  ]
}
```

TP12 – JPQL

Indication pour la méthode JPQL :

Repo:

```
@Query("SELECT distinct (ch) FROM Chambre ch " +
        "INNER JOIN ch.reservations r " +
        "INNER JOIN r.etudiants e " +
        "WHERE e.cinEtudiant=:cin " )
Chambre trouverChselonEt(long cin);
```

Entiy (pour éviter la boucle infinie car relation OneToMany :

```
@OneToMany(cascade = CascadeType.ALL)
@ToString.Exclude
@JsonIgnore
Set<Reservation> reservations;
```

TP12 – JPQL

GET /chambre/trouver-chambre-selon-etudiant/{cin}

Parameters

Name	Description
cin * required string (path)	123321

Execute

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8089/tpfoyer/chambre/trouver-chambre-selon-etudiant/123321' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8089/tpfoyer/chambre/trouver-chambre-selon-etudiant/123321
```

Server response

Code	Details
200	Response body

```
{
  "idChambre": 2,
  "numeroChambre": 105,
  "typeChambre": "SIMPLE",
  "bloc": null
}
```

GET /chambre/trouver-chambre-selon-etudiant/{cin}

Parameters

Name	Description
cin * required string (path)	111111

Execute

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8089/tpfoyer/chambre/trouver-chambre-selon-etudiant/11
  -H 'accept: */*'
```

Request URL

```
http://localhost:8089/tpfoyer/chambre/trouver-chambre-selon-etudiant/1111
```

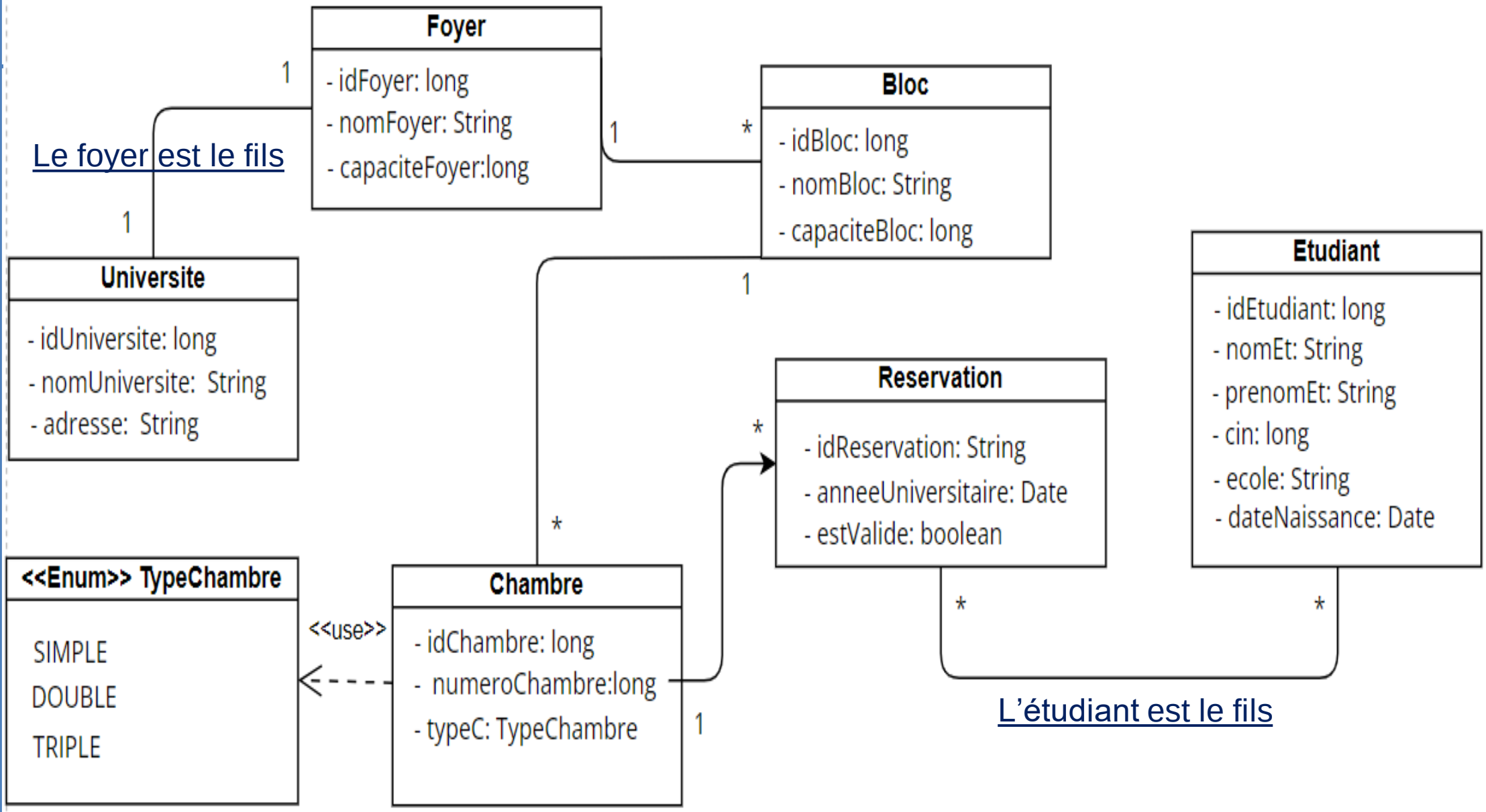
Server response

Code	Details
200	Response headers

```
connection: keep-alive
content-length: 0
date: Sun,26 Nov 2023 14:20:35 GMT
keep-alive: timeout=60
```

TP12

- **Quand utiliser les Keywords et quand utiliser le JPQL?**

Il y a plusieurs cas, nous en parlerons en classe.

Diagrammes de Classes